IaCSecBench: A Finding-Normalization Methodology and Reproducible Harness for Evaluating Infrastructure-as-Code Security Validation

Manideep Chittineni

Received: date / Accepted: date

Abstract

Context. Infrastructure as Code (IaC) is the dominant mechanism for provisioning cloud platforms, and pre-deployment security validation is fragmented across static scanners, policy engines, and infrastructure test frameworks. Comparing these tools is impeded by two obstacles: they operate at incompatible abstraction levels, from lexical scanning of source templates to policy evaluation over resolved execution plans, and each reports findings in a bespoke schema whose rule identifiers carry no shared semantics. Benchmarks authored by tool developers compound the difficulty, because the same reading of a security control tends to define both the policy and the ground truth. **Objective.** We ask how much the evaluation procedure itself, meaning what counts as a correct detection and which cases are admitted, determines the conclusions such a comparison reaches. **Method.** We present IaCSecBench, an open harness that normalizes heterogeneous scanner output onto a canonical control taxonomy before any confusion matrix is formed, admits cases only through a mechanically enforced procedure, and scores detection at three explicit matching criteria. Over 48 admissible cases (26 vulnerable, 22 compliant) we evaluate three third-party scanners spanning two independent rule sets, policy evaluation over compiled plan documents, and a repository-edge layer, reporting exact Clopper–Pearson intervals and exact binomial McNemar tests under Holm–Bonferroni correction. **Results.** The matching criterion moves apparent recall by 7.7 to 11.5 percentage points among the source-level and plan-level tools, which is the same magnitude as the entire spread between those tools, and it reorders them; on the repository-edge layer it moves recall by 88.5 points. No pairwise difference between scanners survives correction, and for three of the ten pairs no possible

outcome could have reached significance at this many discordant cases, which we report rather than describing those tools as equivalent. A third-party scanner, not the pipeline this paper contributes, attains the highest point-estimate recall. **Conclusion.** A scanner comparison that does not state its matching criterion and its admissibility rule is not reproducible at any corpus size. The corpus, normalization engine, execution harness, and analysis code are released as a replication package in which every reported table is a generated artefact.

Keywords Infrastructure as Code Security validation Benchmarking methodology Static analysis Policy as code Empirical evaluation

1 Introduction

Cloud platforms are increasingly provisioned through Infrastructure as Code (IaC), in which infrastructure topology is expressed declaratively in version-controlled artefacts [Morris, 2020, Guerriero et al., 2019], atop service models whose elasticity makes such automation a practical necessity [Armbrust et al., 2010]. The property that makes IaC attractive, deterministic and repeatable application of a declared state, also amplifies its failure modes: a single insecure declaration, such as an unrestricted ingress rule or a disabled encryption setting, propagates automatically to every environment that consumes the module. Empirical studies of configuration repositories confirm that such defects are common, recurring as hard-coded secrets, excessive privileges, and insecure defaults [Rahman et al., 2019b, 2021, Verdet et al., 2023, 2025]. Rahman et al. [2019a] map the resulting research area and identify tooling and empirical evaluation as two of its four constituent topics.

Because remediation after provisioning implies a window of exposure, contemporary practice shifts validation earlier, into continuous integration [Bass et al., 2015, Humble and Farley, 2010]. Doing so raises an evaluation problem the literature has not resolved. Candidate gates operate at incomparable abstraction levels: lexical scanning of HCL source, abstract-syntax-tree analysis, and policy evaluation over resolved plan state. Each reports findings in a bespoke schema, with rule identifiers that carry no shared semantics. Comparative claims are consequently difficult to reproduce, and when a benchmark is authored by the developer of one of the tools under comparison, agreement between that tool and the ground truth is close to guaranteed by construction.

This paper addresses the evaluation problem; it does not propose a further analyser. Three commitments shape the design.

First, *normalization before comparison.* A canonical control taxonomy mediates between native rule identifiers and benchmark ground truth, so that a finding is credited when it identifies the control a case was authored to violate rather than merely because the tool emitted something. Findings whose rule identifiers the taxonomy does not cover are counted and reported rather than discarded, since silently treating an unmapped detection as a miss biases the comparison toward whichever tool the taxonomy was authored around.

Second, *mechanically enforced admissibility.* A case is admitted only if it is valid HCL2 that references resource types the declared provider defines, carries a non-contradictory ground-truth label, and resolves to an unambiguous canonical control. The reported corpus size is the admissible count, not the number of entries in a catalogue.

Third, *measurement separated from assumption.* A tool that is not installed in the measurement environment is reported as such and excluded; it is never represented by an assumed detection rate. Latency is sampled repeatedly and reported with dispersion.

The design follows the benchmarking requirements Hasselbring [2021] sets out for software engineering research, in particular that a benchmark publish its admission rule and its measurement configuration rather than only its headline scores.

1.1 Contributions

- A finding-normalization methodology and canonical control taxonomy that render heterogeneous IaC scanner output commensurable, with three explicit matching strictness levels that separate correct attribution from incidental detection, and a measurement of how far the choice among them moves the conclusion.

- A corpus admissibility procedure that enforces the stated inclusion criteria mechanically, together with the measured admissibility of the released corpus, reported per collection.

- An execution harness that records raw tool output, resolved tool versions, and repeated latency samples, and that refuses to emit results when no case is admissible. Applied to its own reference pipeline, the harness reports that a third-party scanner detects more of the corpus than the pipeline's scored layers do, which is the outcome we publish.

- A statistical protocol appropriate to paired, small-sample scanner comparison: exact Clopper–Pearson intervals [Clopper and Pearson, 1934], the exact binomial form of McNemar's test

[McNemar, 1947, Dietterich, 1998], Holm–Bonferroni correction [Holm, 1979], and a per-comparison statement of what difference the test could have detected at all.

- An open replication package containing the corpus, taxonomy, harness, and analysis code, in which every reported table is a generated artefact.

2 Background and Related Work

2.1 Security Weaknesses in Infrastructure as Code

Rahman et al. [2019b] established that infrastructure scripts contain recurring security smells, first across Puppet and subsequently in Ansible and Chef [Rahman et al., 2021], and derived a defect taxonomy for IaC scripts [Rahman et al., 2020]. Sharma et al. [2016] had earlier catalogued configuration smells over 4,621 Puppet repositories, establishing the maintainability counterpart to the security work. Complementary studies characterise misconfiguration in Kubernetes manifests [Rahman et al., 2023, Shamim et al., 2020], security practice in Terraform repositories specifically [Verdet et al., 2023, 2025], and practitioner conventions drawn from grey literature [Kumara et al., 2021]. Quality and defect-prediction metrics for infrastructure code have also been catalogued [Dalla Palma et al., 2020, 2022]. This body of work motivates automated pre-deployment validation but does not address how competing validation tools should be compared.

2.2 Static Analysis of IaC and Existing Comparisons

Chiari et al. [2022] survey static analysis for IaC and note the absence of shared evaluation infrastructure. The closest detector-side prior art is GLITCH, which detects security smells across several IaC technologies via an intermediate representation and is evaluated against manually annotated oracle datasets with a tool comparison [Saavedra and Ferreira, 2022, Saavedra et al., 2023]. The contrast with this work is a clean one: GLITCH normalizes *inputs*, so that one detector can analyse five languages, whereas IaCSecBench normalizes *outputs*, so that detectors reporting incommensurable findings can be scored against one ground truth. Section 5.5 states why GLITCH's oracles are not the corpus used here and what would be required to use them. The taxonomy such detectors are scored against is itself under revision, most recently beyond the original seven smell categories [War et al., 2025], which is one reason this paper versions its control set as data rather than fixing it in prose.

Comparative evaluation of the practitioner scanners has begun to appear. Roe et al. [2026] benchmark IaC scanners across multiple cloud providers and report per-tool detection counts; Gokhale et al. [2026] contrast policy-driven evaluation with taint-aware static reasoning for IaC misconfiguration; and Kapetanidou et al. [2025] evaluate the equivalent tool class for Kubernetes. These establish that the comparison is wanted. None addresses the two obstacles this paper targets: findings are compared in whichever schema each tool emits, so a rule that fires for the wrong reason is indistinguishable from one that fires for the right one, and the corpora are not released with an admissibility procedure a reader can re-run. We also note that no comparative evaluation of IaC security scanners has yet appeared in an indexed software engineering journal or conference, which bounds how firmly the state of practice can be characterised; our contribution is orthogonal to these studies and could be applied to their corpora.

Opdebeeck et al. [2023] show that control- and data-flow analysis materially changes security-smell detection in IaC [see also Opdebeeck et al., 2022]. That result motivates the plan-level layer evaluated here: a compiled Terraform plan is a resolved artefact in which interpolation, conditionals and expansion have already been applied, so evaluating it sidesteps a class of analysis difficulty instead of solving it. Qiu et al. [2024] pursue the complementary route of recovering semantic checks over IaC programs directly, and policy-violation analysis has been extended to packaged Kubernetes artefacts [Minna et al., 2026]. Empirical work on policy-as-code adoption indicates which controls practitioners actually enforce [Foalem et al., 2026].

2.3 Evaluation Methodology

Methodologically this paper follows the static-analyser evaluation tradition established by Habib and Pradel [2018], who show that headline detection rates are highly sensitive to what counts as a match, and the benchmark-construction cautions of the OWASP Benchmark project [OWASP Foundation, 2023]. The three-level matching criterion of Section 3.4 is a direct response to the first. Our reporting conventions are those of the SAST tool-comparison literature, which reports per-category rather than pooled effectiveness and separates the absence of a rule from the failure of one [Li et al., 2023, Esposito et al., 2024]. Böhme et al. [2022] establish the wider point that benchmark results fail to replicate when configuration and variance go unreported, which is the argument for the version manifest of Section 4.5 and for preferring interval width to corpus size as an indicator of evidential strength. Threats to validity are organised under the categories of Wohlin et al.

[2012], and the study is reported against the empirical standards for software engineering research [Ralph, 2021, Hasselbring, 2021].

2.4 Policy as Code and Native Infrastructure Testing

Open Policy Agent provides general-purpose policy evaluation over structured documents [Open Policy Agent, 2024], and Terraform provides a native test harness for module behaviour [HashiCorp, 2024]. Compliance requirements are drawn from the CIS Amazon Web Services Foundations Benchmark [Center for Internet Security, 2024] and NIST SP 800-53 [National Institute of Standards and Technology, 2020]. IaCSecBench composes these mechanisms; it does not extend them.

3 Framework Architecture

IaCSecBench composes three validation layers, each addressing a distinct failure class. Fig. 1 shows their arrangement and the gating relation between them: a commit reaches a later layer only by passing every earlier one, so the layers compose as a conjunction in execution.

Detection, however, is scored as a union, and the two are consistent. Under gating, a case that any layer would catch is a case the pipeline catches, because the commit is stopped at the first layer that objects; the conjunction governs which layers get to run, not which defects get caught. Where this paper reports the pipeline's detection effectiveness, the figure is therefore the union of layers 1 and 3, marked as such.

Two of the three layers are scored per benchmark case. Layer 2 validates module behaviour rather than detecting resource misconfiguration, so it has no detection count on a corpus of misconfigured resources and is reported by suite size alone in Section 5.3.

3.1 Layer 1: Repository-Edge Data Validation

The first layer prevents sensitive material from entering version control through schema verification, detection of sensitive fields, and pattern-based identification of personal data. It operates on repository content rather than infrastructure semantics, and its characteristic failure mode is over-detection: high-entropy strings in output declarations are frequently flagged as credentials. Over-detection is not a cosmetic cost. Developers discount tools whose warnings they learn to distrust, an effect documented both for static analysis generally [Vassallo et al., 2019] and for security warnings specifically [Tahaei et al., 2021], to

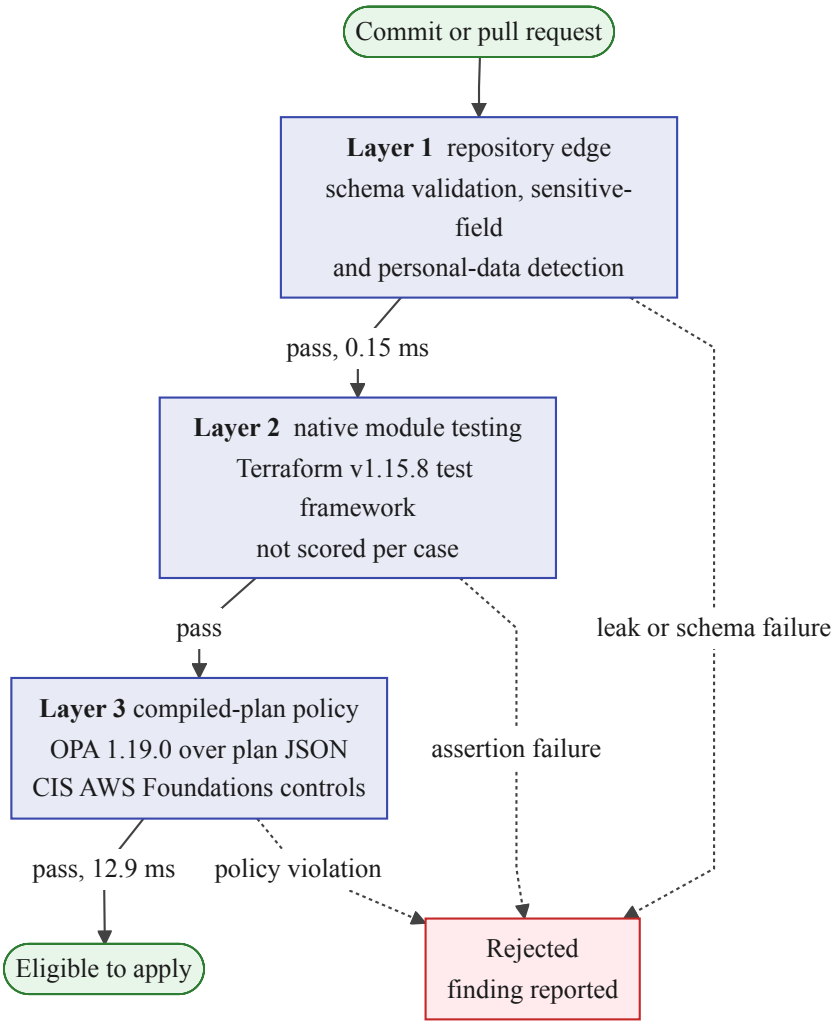

Figure 1: Layered validation pipeline: repository-edge data validation, native module testing, and policy evaluation over compiled plan documents. A change reaches a later layer only by passing every earlier one. Edge labels give the mean per-case latency of the scored layers; the full figures, including dispersion, are in Table 7. Layer 2 validates module behaviour rather than resource configuration, so it carries no detection count on this corpus and is reported by suite size in Section 5.3.

the point that identifying which warnings merit action is now a research area in its own right [Ge et al., 2023]. This layer's false-positive behaviour is therefore reported separately in Section 5 and is not ag-

gregated with the policy layers.

3.2 Layer 2: Native Module Testing

The second layer validates module behaviour using Terraform's native test framework [HashiCorp, 2024], covering variable constraints, output correctness, conditional resource creation, and tagging requirements without provisioning cloud resources.

3.3 Layer 3: Policy Evaluation over Compiled Plans

The third layer converts a Terraform plan to its JSON representation and evaluates Rego policies against it [Open Policy Agent, 2024]. Because the plan is produced after expression resolution, policies observe concrete attribute values rather than unevaluated references. Plan generation is configured so that provider credential validation is disabled, which permits evaluation in continuous integration without cloud account access; this is a precondition for the layer being usable at all, and is implemented by an injected provider override, never by relaxing a policy.

3.4 Finding Normalization

Comparing heterogeneous scanners requires a canonical representation. A finding is normalised to

$$\mathcal{F} = \langle c,\ t,\ \kappa,\ r,\ s \rangle, \tag{1}$$

where c identifies the benchmark case, t the tool, κ the set of canonical controls the tool's native rule maps to, r the resource address, and s the reported severity. The mapping from native rule to κ is many-to-many: a coarse rule may legitimately cover several controls, and such rules are reported explicitly because they weaken attribution precision.

Detection is then evaluated at three strictness levels:

- *control*: some finding's κ intersects the controls the case was authored to violate. This is the primary criterion.
- *resource*: some finding names the resource address the ground truth identifies, irrespective of which rule fired. This isolates the correct-resource-wrong-reason case.
- *any*: the tool emitted any finding within the case. This criterion overstates detection and is reported to quantify how much of a tool's apparent performance is incidental.

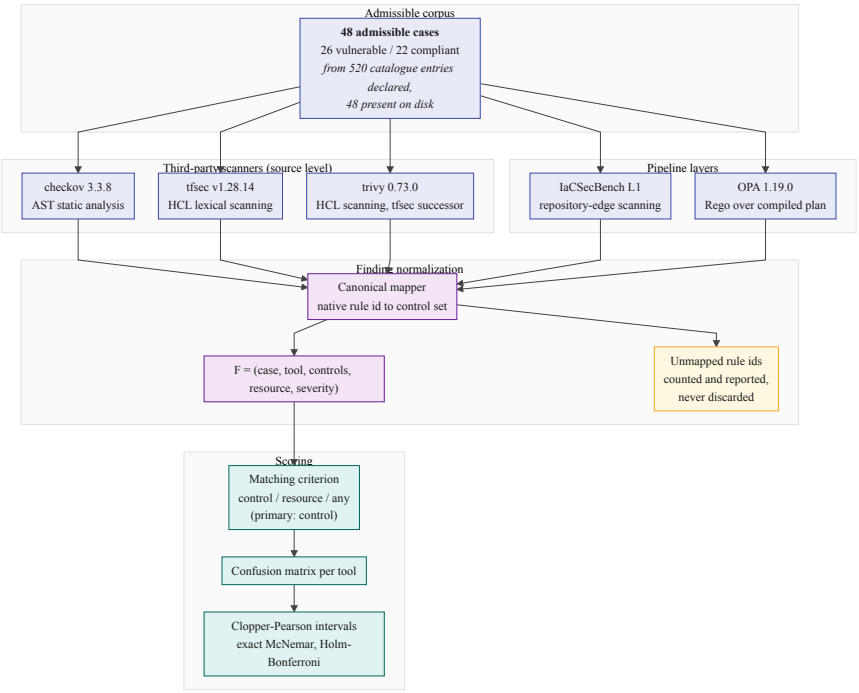

Figure 2: Finding-normalization engine and evaluation pipeline: heterogeneous scanner output is mapped onto the canonical control taxonomy before any confusion matrix is formed.

Reporting all three is deliberate: adopting a single permissive criterion is the most common mechanism by which scanner benchmarks overstate detection [Habib and Pradel, 2018]. Where the ground truth does not name a resource address, the resource criterion is inapplicable and is reported as such.

Fig. 2 traces a finding through the engine. The ordering matters for the comparison's validity: normalization onto the canonical taxonomy precedes confusion-matrix construction, so no tool is scored in its own reporting schema and the matching criterion is applied identically to every tool.

4 Evaluation Methodology

The threat classes the pipeline's controls were selected against, and the four domains held out of scope, are given in Appendix A. That decomposition delimits intent; the operative statement of what this evaluation measured is the corpus coverage of Section 4.2.

Table 1: Corpus admissibility. The admissible count is the corpus size reported throughout; catalogue entries without configuration are not cases. The declared and present counts are given per collection because the two collections diverge by very different factors, and the external collection is the one the generalization analysis rests on. Rejection reasons are determined mechanically by `evaluation/corpus.py`.

Corpus status	**Internal**	**External**	**Total**
Catalogue entries declared	345	175	520
Configurations present on disk	44	4	48
Admissible	**44**	**4**	**48**
vulnerable	22	4	26
compliant baseline	22	0	22

4.1 Corpus Construction and Admissibility

The corpus separates a controlled collection, authored for this evaluation, from an external collection sourced independently. The distinction matters because the controlled collection shares an author with the policy set and therefore cannot provide independent confirmation of detection capability.

Admissibility is enforced mechanically. A case is admitted only if it satisfies all of the following:

1. it declares at least one resource or module block;
2. its ground-truth label is present and internally consistent, where a case declaring contradictory labels is rejected rather than resolved by precedence;
3. it resolves to exactly one interpretation in the canonical taxonomy, since a CIS section number may span several distinct controls and inference cannot choose between them;
4. `terraform init` and `terraform validate` succeed, which is the only check that detects a case referencing a resource type the provider does not define.

Table 1 reports the outcome, per collection. The admissible count is the corpus size used throughout; catalogue entries without corresponding configuration are not cases and are excluded. The two collections diverge by very different factors, which is why the table is split: the controlled catalogue declares 345 entries against 44 configurations, and the external catalogue 175 against 4.

Both gaps require explanation, and they have different causes. The external manifests enumerate every example in the source benchmark documents, most of which were never transcribed into runnable configurations; a manifest entry without a configuration is a citation rather than a case, and counting such entries would inflate the external collection by more than a factor of forty while contributing no measurement. The controlled catalogue is the residue of a larger planned corpus: it enumerates the per-control specifications the generator is capable of emitting, of which 44 were generated, validated and admitted. We report the declared count rather than pruning the catalogue to match, because the gap is itself the datum that motivates reporting admissible rather than catalogued size, and because the specifications indicate where the corpus can be extended. No reported figure derives from a declared-but-absent entry.

4.2 Control Coverage

The canonical taxonomy defines 26 controls, of which 22 are exercised by at least one admissible case. The two numbers answer different questions: the size of the taxonomy describes what the normalization engine can express, while the exercised count describes what this evaluation measured. Reporting only the former would credit the corpus with four controls, one for API-gateway authorization and three for Kubernetes workloads, against which nothing was ever run.

The unexercised controls are concentrated in the domains the corpus does not reach at all: Kubernetes workloads and serverless authorization. Their absence is a coverage limitation of the corpus rather than a negative result about any tool, and no tool is penalised for them, since a control with no case contributes to no confusion matrix.

Auditing the taxonomy against the installed tool catalogues found three defect classes in the mapping itself, all corrected. Seven controls named plan-level policy rules the policy set does not implement. Two named Checkov identifiers absent from the installed catalogue entirely, and one mapped a resource-*requests* check to a resource-*limits* control, which are distinct Kubernetes concepts. None of these can produce a spurious detection, because nothing emits the identifier; each instead inflates the apparent coverage of the layer it is attributed to, and in the first case that layer is the reference implementation. That is the direction of error that flatters the contribution under evaluation, which is why the audit was mechanical, and why the resulting invariant, that every plan-level identifier must exist in the policy source, is now enforced by a test.

The correction left two controls with no implementing rule in any

tool, and so detectable by none of them. A control in that state scores every case citing it as a miss for every tool simultaneously, which reads as unanimous tool failure rather than as absent coverage; both were removed, and neither was exercised, so no reported figure changes. Eight controls retain an unverified flag, each naming the single identifier behind it. All eight are tfsec long identifiers that this environment cannot corroborate: tfsec 1.28 provides no check-listing command, and its `--filter-results` option returns nothing for an unknown identifier and for a real but untriggered one alike. They are retained rather than deleted, because a wrong identifier is inert and the only possible effect of keeping it is to credit a comparator if it turns out to be real. Where the two options differ we take the one that cannot flatter the reference.

4.3 Ground-Truth Labelling

Labels are derived mechanically rather than by human rating. Each controlled case is emitted by a generator from a per-control specification that declares the intended violation, and the label follows from which member of the vulnerable/compliant pair is being written. The generator, the specification, and the emitted cases are all in the replication package, so any reader can check that a case carries the label its specification declares.

Because labels are a deterministic function of the specifications, rater disagreement is impossible by construction, and the threat that replaces it is *specification error*: a specification that misreads its originating control produces a wrong label reproduced identically across the whole pair. An agreement coefficient computed over the generator's own output would measure its determinism, not concurrence, so we do not report one.

We instead ran a blind second labelling pass against the threat that does apply. **The second rater is a large language model, not a second human investigator**, and that choice bounds the claim in ways set out below. For each admissible case a review view was constructed containing only the Terraform configuration and the canonical control under test, with every comment line removed, since the generator writes the expected label and its rationale into each file header, and with the annotation files and the case identifier withheld, the latter because its suffix names the class. Cases were presented in an order keyed on a hash of the case identifier, and the views were screened for residual label vocabulary. Each was then labelled from the configuration and the control text alone, with a reason recorded before any comparison.

Agreement is reported before reconciliation, since resolving disagreements guarantees consensus: 47 of 48 cases agreed, giving raw agreement of 97.92% and Cohen's $\kappa = 0.958$ against a chance-agreement baseline of 50.17%. With this class balance, 26 vulnerable against 22 compliant, expected agreement is close to one half, so κ here carries little information beyond the raw count; the 2×2 agreement table is in the replication package.

The single disagreement is the kind of defect this pass exists to surface. A bucket configured with SSE-S3 (`AES256`) was labelled violating under a control titled *lacks server-side encryption* and citing CIS Amazon Web Services 2.1.1. That configuration does not lack server-side encryption, and CIS 2.1.1 is satisfied by SSE-S3; read against its stated text the case is compliant. The generator's rationale reveals the intent, namely encryption *other than* a customer-managed key, so the label encoded a requirement the control text never stated. That stricter requirement is also what the reference policy set enforces, which makes this a concrete instance of the construct-validity threat of Section 7.1 rather than a hypothetical one: the ground truth agreed with the reference implementation rather than with the control it cited.

We corrected the control text rather than the label, because requiring a customer-managed key is a defensible control and is what the corpus, the policy set, and the customer-key rules of both source-level scanners all test; relabelling instead would have left the pair testing nothing. The control is retitled to state the key requirement and records that it is stricter than the CIS control it cites. No confusion-matrix entry changed. A regression test now pins the citation in the taxonomy to the one in the generator specification so the two cannot drift apart silently.

Two properties bound what this pass establishes. The second rater is automated, so this is not human inter-rater reliability; and its reading of the control texts derives from the same public documentation the specifications were written from, so the two raters are not fully independent and agreement is biased upward accordingly. Five cases had been inspected during earlier debugging and their agreement is not blind; excluding all five leaves 42 of 43 agreeing, with the disagreement outside that set. The full per-case labels, reasons, and these caveats are in the replication package.

Two further checks bear on specification error. The admissibility procedure of Section 4.1 rejects any case whose label is absent, internally contradictory, or resolvable to more than one control in the taxonomy. And the audit of the control map reported in Section 4.2 found three classes of mapping defect by mechanical comparison against the installed tool catalogues. Neither check can establish that a specifica-

tion reads its control correctly; that remains an unmitigated threat, and independent relabelling by a second investigator is the obvious next step.

4.4 Research Questions

The matching criterion is the primary object of study, and the research questions are ordered accordingly.

- **RQ1**: How much does the matching criterion change apparent detection, relative to the differences between the tools being compared?
- **RQ2**: What detection effectiveness do the evaluated tools achieve on the admissible corpus, how precisely is that effectiveness estimated, and what difference could the comparison have detected?
- **RQ3**: What execution latency does each layer contribute in continuous integration, and what is the size of the native validation suite required to express the corresponding assertions?
- **RQ4**: Which layers account for which detections?
- **RQ5**: Does effectiveness persist on independently sourced configurations?

4.5 Measurement Environment and Tool Versions

Tool versions are resolved at execution time and recorded in `results/run_manifest.json`, together with the platform, processor, and repository commit. Versions are not restated in prose, because a hand-copied version string is the first thing to drift; the manifest is authoritative [Böhme et al., 2022].

Tools not installed in the measurement environment are recorded as such and omitted from every table. No tool is assigned an assumed detection rate or an assumed latency.

4.6 Tool Selection

The comparison covers Checkov, tfsec, Trivy, and standalone policy evaluation over plan documents. Selection was governed by two criteria: the tool must parse HCL2 or Terraform plan JSON without requiring cloud provider API access, and the set must span distinct analysis paradigms, namely syntax-tree analysis, lexical scanning, and policy evaluation. KICS, Regula, and provider-native configuration

services are excluded under the first criterion or as duplicating a represented paradigm.

Trivy requires separate comment, because it is in the set for a reason unrelated to paradigm coverage. Aqua Security folded tfsec's engine and rule set into Trivy, and Trivy is the maintained product; tfsec is no longer developed. A comparison reporting tfsec alone would be characterising a scanner practitioners are advised off, so both are measured. But the two are not independent implementations. Their rule identifiers are the same Aqua Vulnerability Database numbers under different spellings (tfsec reports `AVD-AWS-0086` where Trivy reports `AWS-0086`), and we exploit that correspondence to build Trivy's entries in the control taxonomy: each is derived mechanically from the identifier pair tfsec itself emits, rather than chosen by observing which rules Trivy happens to fire on a case, which would fit the taxonomy to the tool. A regression test re-derives the crosswalk and fails if the two disagree.

The consequence bounds what the enlarged tool set establishes. Three third-party scanners are measured, but only *two* independent third-party rule sets: Checkov's, and the tfsec/Trivy lineage. Agreement between tfsec and Trivy is therefore close to a consistency check on that shared lineage across two engine generations rather than corroboration by a second party, and we do not report it as the latter. Where the two diverge, the difference is attributable to engine and rule evolution between the products, and is informative for that reason.

Terrascan is absent for a reason that is not a selection criterion, and we state it plainly: no successful Terrascan execution was ever recorded against this corpus, so there is nothing to report. Its paradigm, Rego policy evaluation, is represented in the tool set by the plan-level layer, which bounds what the omission costs without eliminating it. Policy evaluation is therefore represented only over compiled plan documents, and no source-level policy engine appears in the tool set. A source-level Rego engine could differ from a plan-level one on configurations whose violations are resolved during planning, and this evaluation does not measure that difference. We make no claim of any kind about Terrascan's detection behaviour.

5 Results

All tables in this section are generated by `evaluation/analyze.py` from recorded raw tool output, regenerated by `experiments/run_baselines.sh`, and not edited by hand.

Table 2: Recall under three finding-matching criteria. *Control* credits a detection only when the tool's rule maps to the control the case violates; *resource* credits any finding on the expected resource; *any* credits any finding whatsoever. Δ is the control-to-any spread in percentage points, which measures how much of a tool's apparent detection is incidental. It is given per tool rather than summarised, because the spread differs by an order of magnitude across the set and a single headline figure would be carried entirely by its largest member.

Tool	**Control (%)**	**Resource (%)**	**Any (%)**	**Δ (pts)**
Checkov	88.46	76.92	96.15	7.69
IaCSecBench L1	3.85	0.00	92.31	88.46
OPA (plan-level)	76.92	65.38	84.62	7.69
tfsec	84.62	73.08	92.31	7.69
Trivy	80.77	69.23	92.31	11.54

5.1 RQ1: Sensitivity to the Matching Criterion

Table 2 reports recall under each criterion, with the control-to-permissive spread Δ given per tool. Two readings of that column matter, and they differ in magnitude by an order of magnitude.

Among the three source-level scanners and the plan-level layer, Δ ranges from 7.69 to 11.54 percentage points. The spread between those same four tools at the control criterion is 11.54 points, from Checkov's 88.46 to the plan-level layer's 76.92. The criterion effect and the between-tool effect are therefore of *the same magnitude* on this corpus, and the criterion is enough to reorder the tools: under the permissive criterion tfsec, Trivy and the repository-edge layer are indistinguishable at 92.31 while Checkov leads at 96.15, an ordering that the control criterion does not produce. A comparison that reports a single detection rate without stating its criterion has therefore not established the ranking it appears to report.

On the repository-edge layer Δ is 88.46 points, from 3.85 at the control criterion to 92.31 under the permissive one. That layer detects hardcoded credentials and comparable textual patterns, so on a corpus dominated by attribute-level controls it identifies the seeded violation in exactly one case while emitting some finding in almost all of them. The ratio between its two figures is close to twenty-four, and we report the difference in points rather than that ratio: with one true positive in the denominator the ratio is an artefact of a small count, whereas the 88-point difference is the quantity that would mislead a reader of a single-criterion report.

Table 3: Confusion-matrix counts on the admissible corpus ($N = 48$: 26 vulnerable, 22 compliant). Rates and intervals over the same counts are in Table 4.

Tool	TP	FP	TN	FN	MCC
Checkov	23	0	22	3	0.882
IaCSecBench L1	1	1	21	25	-0.017
OPA (plan-level)	20	0	22	6	0.777
tfsec	22	2	20	4	0.753
Trivy	21	1	21	5	0.762

Table 4: Detection rates with exact Clopper–Pearson 95% intervals, over the counts of Table 3. Recall's denominator is the vulnerable-case count fixed by the corpus design; precision's is the number of findings the tool emitted, so its interval is conditional on that realised count rather than a marginal statement about the tool. Entries marked n/e are not estimable from this corpus.

Tool	Recall %	Precision %	Specificity %
Checkov	88.46 [69.85, 97.55]	100.00 [85.18, 100.00]	100.00 [84.56, 100.00]
IaCSecBench L1	3.85 [0.10, 19.64]	50.00 [1.26, 98.74]	95.45 [77.16, 99.88]
OPA (plan-level)	76.92 [56.35, 91.03]	100.00 [83.16, 100.00]	100.00 [84.56, 100.00]
tfsec	84.62 [65.13, 95.64]	91.67 [73.00, 98.97]	90.91 [70.84, 98.88]
Trivy	80.77 [60.65, 93.45]	95.45 [77.16, 99.88]	95.45 [77.16, 99.88]

5.2 RQ2: Detection Effectiveness and Resolvable Differences

Table 3 reports confusion-matrix counts and Table 4 the corresponding rates with exact Clopper–Pearson intervals. Intervals accompany every proportion because a point estimate on a corpus of this size is compatible with a wide range of true values; the two tools attaining 100.00% precision carry lower confidence bounds of 85.18 and 83.16 respectively, and reporting the point estimate alone would misstate what has been established.

The Clopper–Pearson form is chosen for its coverage guarantee rather than for interval width. Its actual coverage is never below the nominal level, whereas the score and adjusted-Wald intervals attain nominal coverage only on average and undercover for proportions near the boundary [Agresti and Coull, 1998], which is where most of these estimates sit. The cost is conservatism, so every interval in Table 4 is wider than a score interval on the same counts; for a claim about

Table 5: Exact McNemar comparisons against OPA (plan-level). b counts cases the reference classifies correctly and the comparator does not, counting both missed violations and spurious findings; c is the converse. p is the two-sided exact binomial value, p_{adj} its Holm–Bonferroni correction across the tool family. The odds ratio carries a Haldane–Anscombe correction so that a zero discordant cell yields a finite estimate. $d_{\min}$ is the smallest $|b - c|$ the exact test could have called significant at this many discordant pairs; a dash means no split could, so the comparison had no power and its non-significance is not evidence of similarity.

Comparator	b	c	$d_{\min}$	p	p_{adj}	**OR**	Cohen's g
Checkov	2	5	7	0.453	1.000	0.45	0.214
IaCSecBench L1	21	1	12	< 0.001	< 0.001	14.33	0.455
tfsec	3	3	6	1.000	1.000	1.00	0.000
Trivy	3	3	6	1.000	1.000	1.00	0.000

detection capability that is the safe direction of error, and the harness computes the score interval as well, so a reader preferring it can obtain it from the released results.

One property of the precision intervals should be read with care. Recall's denominator is the 26 vulnerable cases, fixed by the corpus design, so a binomial interval applies directly. Precision's denominator is the number of findings the tool emitted, which is a realised outcome rather than a design parameter, so its interval is conditional on the observed prediction count and is not a marginal statement about the tool.

Specificity is estimable here because the corpus carries 22 ground-truth compliant cases, and it is reported for that reason: a detector's cost to its users is carried as much by its false positives as by its misses, and a table reporting only recall and precision would have described the half of the behaviour that flatters a detector. The Matthews correlation coefficient [Matthews, 1975] is likewise estimable and is given in Table 3. What the compliant baseline does not yet support is a *precise* false-positive estimate: with 22 negatives the specificity intervals span roughly fifteen points even where no false positive occurred.

Tables 5 and 6 report pairwise comparisons. Table 6 is the confirmatory analysis: it covers all ten unordered pairs with Holm–Bonferroni applied across the whole set, so the comparison a practitioner most wants, between two third-party scanners, is made, and the tool this paper contributes does not sit on one side of every test. Table 5 retains the against-one-reference form for continuity with the

Table 6: Exact McNemar comparisons over every tool pair, with Holm–Bonferroni applied across the full set of pairs rather than across one tool's family. b counts cases the left tool classifies correctly and the right does not; c is the converse. Both error types count. The correction is stronger here than in Table 5 because the family is larger. $d_{\min}$ is the smallest $|b - c|$ this pair's discordant count could have resolved; a dash means none could.

Tool A	**Tool B**	b	c	$d_{\min}$	p	p_{adj}
Checkov	IaCSecBench L1	24	1	11	< 0.001	< 0.001
Checkov	OPA (plan-level)	5	2	7	0.453	1.000
Checkov	tfsec	4	1	—	0.375	1.000
Checkov	Trivy	4	1	—	0.375	1.000
IaCSecBench L1	OPA (plan-level)	1	21	12	< 0.001	< 0.001
IaCSecBench L1	tfsec	3	23	12	< 0.001	< 0.001
IaCSecBench L1	Trivy	2	22	12	< 0.001	< 0.001
OPA (plan-level)	tfsec	3	3	6	1.000	1.000
OPA (plan-level)	Trivy	3	3	6	1.000	1.000
tfsec	Trivy	1	1	—	1.000	1.000

way this literature usually reports, and its weaker correction reflects its smaller family; where the two disagree, the all-pairs adjustment governs.

The discordant count b includes every case in which the first tool is correct and the second is not, counting spurious findings as well as missed violations; restricting b to missed violations understates disagreement. The exact binomial p-value is reported in preference to the chi-square approximation, which is unreliable when a discordant cell is small; on these counts, where the smallest discordant cell is one, the approximation is not merely conservative but invalid. Test selection follows established guidance for software engineering experiments [Arcuri and Briand, 2014]. Odds ratios carry a Haldane–Anscombe correction, so a zero discordant cell yields a finite estimate; an unbounded ratio is a degenerate cell rather than a large effect, and Cohen's g is reported as the effect size.

No pairwise difference among the three third-party scanners and the plan-level layer is significant after correction. The $d_{\min}$ column states what each comparison could have detected, and it is the more informative result. For Checkov against the plan-level layer, seven discordant cases mean that only a seven-to-zero split would have reached $\alpha = 0.05$; the observed split was five to two. For three pairs, Checkov against tfsec, Checkov against Trivy, and tfsec against Trivy, the discordant count is five or fewer, at which *no* split of the discordant cases

reaches significance: the smallest attainable two-sided p-value at five discordant pairs is 0.0625. Those three comparisons had no power at all, and their non-significance carries no information about the tools whatsoever. Describing any of these tools as equivalent on this evidence would be a misreading of a test that could not have concluded otherwise.

The one significant family is the repository-edge layer against each of the other four, which reflects that it addresses a different class of defect entirely; that is a statement about scope, not quality. The interval widths in Table 4 and the $d_{\min}$ column of Table 6 together are the honest summary of what this corpus establishes about relative effectiveness among the scanners, which is little, and they say so in a form a reader can act on.

5.2.1 Divergence within the shared rule lineage

The tfsec/Trivy pair (Section 4.6) permits one measurement the rest of the comparison cannot make: how much a rule set changes as it is carried from a retired product into its successor. The two agree on 46 of 48 cases. Both disagreements concern the same control, IAM policy wildcards: tfsec reports `aws-iam-no-policy-wildcards` on both members of the pair, and Trivy emits its counterpart `AWS-0057` on neither, not on these two cases and nowhere in the corpus.

The divergence is not a straightforward regression. tfsec's rule fires on the violating case, earning a true positive, *and* on the compliant case, earning a false positive; Trivy's silence forfeits both. tfsec therefore records the higher recall and Trivy the higher precision, from a single rule behaving differently. We verified that Trivy's miss is genuine and not an artefact of our taxonomy: the one unmapped finding Trivy emitted on that case is an S3 versioning rule, unrelated to IAM policy wildcards, so no mapping gap is concealing a detection.

The remainder of the two tools' output corroborates the shared provenance quantitatively. Of the rule identifiers each emitted that the taxonomy does not cover, 12 of tfsec's 13 have an exact Trivy counterpart reported on an identical number of cases; the thirteenth, a CloudFront TLS-policy rule, is absent from Trivy as `AWS-0057` is. Two products separated by a major version and a change of engine thus differ, on this corpus, by two rules. Practitioners migrating from the retired scanner to its replacement should not assume that coverage transfers rule for rule, and a benchmark measuring only one of the two would have attributed the difference to nothing.

Table 7: Measured per-case execution latency over 3 repetitions. Values are mean ± standard deviation on the environment recorded in `results/run_manifest.json`. Each tool is measured at its own interface: the plan-level figure covers policy evaluation over an already-compiled plan document and excludes the `terraform init` and `terraform plan` invocations that produce it, which dominate that layer's wall-clock cost in continuous integration. It is therefore a lower bound, and is not comparable with the source-level figures, which have no equivalent preparation step.

Tool	**Evaluation layer**	**Latency (ms)**
Checkov	AST static analysis	1853.42 ± 58.90
IaCSecBench L1	Repository-edge scanning	0.15 ± 0.05
OPA (plan-level)	Rego over compiled plan	12.94 ± 5.79
tfsec	HCL lexical scanning	275.37 ± 8.35
Trivy	HCL scanning, tfsec successor	405.48 ± 31.97

5.3 RQ3: Execution Cost and Suite Size

Table 7 reports per-case latency as a mean with standard deviation over 144 samples per tool, three repetitions across the 48 cases. Single-run figures are not reported: variance on shared-tenancy runners is large enough that a single sample is not a measurement.

On engineering effort we report a measurement of the native suite alone and withdraw the comparative claim. The validation suite comprises 11 `.tftest.hcl` files totalling 190 physical lines, of which 153 are non-blank and 137 are both non-blank and non-comment, expressing 11 `run` blocks and 26 `assert` blocks. We give both line counts because they differ by the 16 comment lines and one of them is the figure a reader would compute; quoting the larger under the stricter description would inflate the suite by 12%. Counts were obtained by line-oriented pattern matching over the suite sources, which are in the replication package so the count can be reproduced or disputed.

We do not report a code-volume ratio against an external test framework. Doing so would require an independently authored equivalent suite, and no such implementation exists for this infrastructure. Constructing one ourselves would place both sides of the comparison in the hands of the party with an interest in the outcome, which is not a measurement we would ask a reader to accept. The claim that native testing reduces test-suite volume relative to an external framework is therefore outside the scope of this paper. We note only that the native suite requires no language runtime, dependency manifest, or build step beyond the Terraform binary already needed to plan the config-

Table 8: Layer attribution over the vulnerable cases. Overlap is reported explicitly: layers are not assumed to detect disjoint sets. Layer 2 (native module testing) is not scored: it validates module behaviour rather than detecting resource misconfiguration, so it has no detection count on this corpus.

Configuration	**Detected**	**Recall (%)**
L1 (repository edge)	1	3.85
L3 (compiled plan)	20	76.92
L1 ∩ L3 (overlap)	0	0.00
Union of scored layers	**21**	**80.77**

uration, which is a qualitative difference in toolchain surface rather than a quantitative claim about effort.

5.4 RQ4: Layer Attribution

Table 8 attributes detections to the layer that produced them. Each layer is scored by the same normalization and matching procedure applied to the external scanners, so a layer is credited only when its finding maps to the control the case seeds; no layer receives credit by construction.

The table reports pairwise intersections and the union alongside the per-layer counts, because the per-layer counts alone cannot distinguish a complementary pipeline from a redundant one. Reported additively, a case detected by every layer would be counted repeatedly and the composed pipeline would appear to achieve more than it does. The union is therefore the only figure that characterises the pipeline as a whole, and any excess of the summed per-layer counts over the union is redundancy rather than additional coverage.

Two structural constraints bound what this attribution can show, and both suppress apparent complementarity. The repository-edge layer is insensitive to every control expressed as a resource attribute, so on this corpus its contribution is necessarily small, as Section 5.1 already showed from the other direction. The plan-level layer can only be credited on cases for which a plan was produced; cases whose planning failed are excluded from its denominator and are not recorded as misses. The native module-testing layer validates the project's own infrastructure and is not exercised per benchmark case, so it is excluded from this table and reported under RQ3; its omission is a scope boundary, not a null result.

5.5 RQ5: External Generalization

Generalization is assessed only on subsets whose configurations are present in the replication package, four in total (Table 1). They are drawn from published CIS Amazon Web Services Benchmark examples and carry ground-truth labels derived from the benchmark control each example illustrates; they enter the admissible corpus on the same terms as the internal cases and are validated by the same procedure.

One defect in how they were stored is worth reporting, because its signature is invisible in aggregate. A case is scanned as a directory rather than as a file, and these four configurations initially shared one directory. Every tool therefore received identical input for all four cases and returned a byte-identical finding set for each, while the scoring stage compared that one result against four different controls. Four rows entered each confusion matrix where one observation existed. This does not add noise so much as misstate precision: interval width is driven by the number of independent observations, so duplicated cases narrow every interval that includes them without contributing evidence, and nothing in the aggregate output distinguishes this from four genuinely independent cases that happen to agree. Each configuration now occupies its own directory, and the corpus loader refuses to load a corpus in which two cases share one. The same trap applies to any evaluation whose unit of analysis is a directory: case isolation has to be enforced rather than assumed.

Four cases are too few for a per-subset recall estimate, whose interval would span most of the unit interval, so we report them within the combined admissible corpus and treat the external subset as a check that the harness operates on configurations it did not generate rather than as an independent effectiveness estimate.

We make no claim that effectiveness measured here transfers to production infrastructure. Establishing that requires a labelled corpus of real-world configurations, and the position of this literature deserves to be stated precisely rather than as an absence. Manually annotated oracle datasets of real-world IaC scripts do exist: GLITCH releases three, covering Terraform among five technologies, validated by the authors and three independent external raters [Saavedra and Ferreira, 2022]. They are not the corpus used here because their labels are security-smell categories in the sense of Rahman et al. [2019b], whereas this evaluation's unit of ground truth is a CIS control identifier, and the two taxonomies are not in correspondence: a single smell category spans several CIS controls and several controls have no smell counterpart. Scoring the tools evaluated here against those oracles would require a mapping between the two taxonomies that does not

exist and whose construction is a research contribution in itself. We identify that mapping, rather than the absence of labelled real-world data, as the principal obstacle to field-representative evaluation in this area, and return to it in Section 9.

6 Discussion

Two findings from building the harness bear on how IaC security tooling should be evaluated.

First, the matching criterion is as consequential as the choice of tool. Because native rule identifiers carry no shared semantics, any comparison implicitly chooses what counts as a correct detection, and on this corpus that choice moves recall by as much as the entire spread between the tools compared, and reorders them (Section 5.1). This restates Habib and Pradel's finding for IaC and parallels the per-category reporting discipline the SAST literature arrived at for the same reason [Li et al., 2023]. A benchmark's matching criterion belongs in its reported method, alongside its corpus size.

Second, taxonomy coverage is a bias mechanism. When a canonical taxonomy is authored around one tool's rule set, findings that other tools emit under identifiers the taxonomy lacks are scored as misses. The effect is systematic and favours the authoring tool. Auditing observed rule identifiers against the taxonomy after execution, and reporting the unmapped remainder, is the minimum control; during this work the audit identified rules from comparator tools that the initial taxonomy omitted and that would otherwise have been scored as failures.

A third observation concerns statistical reporting rather than IaC. On corpora of this size a non-significant paired test is routinely read as evidence of similarity, and for three of the ten pairs measured here no outcome could have been significant at all. The quantity that distinguishes the two readings, the smallest discordant imbalance the test could resolve, is computable from the discordant count alone and costs nothing to report. We suggest it accompany every paired comparison on a small corpus.

Finally, plan-level evaluation is a trade-off rather than an improvement. Because the compiled plan is produced after expression resolution, policies observe concrete values and avoid a class of interpolation-related imprecision that source-level analysis must otherwise reason about [Opdebeeck et al., 2023]. The cost is that plan generation dominates the layer's latency and requires a working provider configuration.

7 Threats to Validity

Threats are organised under the categories of Wohlin et al. [2012].

7.1 Construct Validity

The controlled corpus was authored by the same investigator who implemented the evaluated policy set. Ground-truth labels therefore encode the same reading of each control that the policies encode, and agreement between the two is not independent evidence of detection capability. This is the most serious limitation of the evaluation, and it is only partly mitigated.

Two mitigations apply. Labels derive from the text of the originating control rather than from any policy implementation, and both the specifications and the generator that consumes them are released, so a reader can audit the derivation. And generalization is assessed on configurations the investigator did not author.

A third check bears on it partially. The blind relabelling pass of Section 4.3 agreed with 47 of 48 recorded labels, and the one disagreement it surfaced was a genuine specification defect of exactly the predicted kind: a control whose stated text was weaker than the requirement its label encoded, where the stricter reading matched the reference policy set. That the pass found such a case is evidence the mechanism is real rather than merely conceivable.

It is not a sufficient mitigation, and we do not present it as one. The second rater is a language model, so no claim of human inter-rater reliability is made, and its reading of control texts derives from the same public documentation the specifications were written from, so the two raters share provenance and agreement is biased upward. A single investigator still wrote every specification. The external subset is the only part of the corpus outside this failure mode entirely, and four cases cannot estimate effectiveness. Independent relabelling by a second human who has not read the policy set remains the correct mitigation and remains outstanding.

Results on the controlled corpus should therefore be read as an upper bound on field performance. A high score on a developer-authored corpus is at least as consistent with insufficient corpus difficulty as with strong detection, and we do not interpret it as the latter.

7.2 Internal Validity

Comparator tools ran with default rule sets, whereas the plan-level policies were authored with knowledge of the corpus taxonomy. This asymmetry favours the plan-level layer. It is disclosed rather than

adjusted for, because equivalent tuning effort across heterogeneous engines cannot be operationalised objectively; readers should treat the comparison as bounding what default-configuration scanning achieves, not as a ranking of the tools' attainable capability.

One plan-level policy was corrected after its results had been inspected, and the correction is disclosed because it is exactly the iteration the comparators did not receive. The audit-log-encryption rule tested for the presence of a key-identifier attribute in the plan document. Terraform serialises three configuration states into that document: an attribute set to a literal carries its value; an attribute set from a reference to another resource is omitted and recorded as not-yet-known; and an attribute that is not set at all is present with a null value. A presence test therefore fails on the configuration that omits the attribute and succeeds on the configuration that sets it from a reference, which inverts the control rather than merely weakening it. The compliant case was reported as violating and the violating case as compliant, and the two errors concealed each other in the aggregate: the confusion matrix showed one false positive and one false negative, a signature indistinguishable from ordinary imprecision.

The corrected rule distinguishes not-set from not-yet-known and reports only the former. The latter is genuinely indeterminate at plan time, because the attribute is set but the plan does not record the value it is set to, and it is therefore reported as neither compliant nor violating. This is a property of plan-level evaluation rather than of this implementation: any policy engine reading plan documents inherits the same three-way distinction, and any that collapses it will invert the same class of control. We report it as a finding about the evaluation layer, and note that the same defect class does not affect block-typed attributes, whose unconfigured form is absent from the plan and can be tested for presence safely.

Because this correction was informed by inspecting corpus results, the plan-level figures reflect one round of iteration that Checkov and tfsec did not receive. The pre-correction and post-correction results are both in the replication package.

Latency was measured on a single host with recorded specifications, so absolute figures do not transfer across environments. Latency additionally measures each tool at its own interface: the plan-level figure covers policy evaluation over an already-compiled plan document and excludes the `terraform init` and `terraform plan` invocations that produce it, which dominate that layer's true wall-clock cost in continuous integration. The source-level scanners have no comparable preparation step. Plan-level and source-level latencies are therefore not interchangeable, and the plan-level figure is a lower bound on de-

ployed cost.

7.3 External Validity

The corpus targets Terraform against a single cloud provider, and the native module-testing layer is exercised against one project's infrastructure. Neither the reported effectiveness nor the suite-size measurement of Section 5.3 generalises to other IaC technologies, other providers, or the multi-module configurations typical of large organisations. No claim is made about organisational factors such as policy ownership, escalation workflow, or developer friction, because this study measures tool output and does not observe practitioners.

The tool set bounds generalisation further: three third-party scanners are measured but only two independent rule sets are represented (Section 4.6), so the comparison speaks to those rule sets rather than to source-level IaC scanning in general. Provider coverage bounds it again, since security-policy adoption in Terraform differs materially across cloud providers [Verdet et al., 2025] and this corpus targets one.

7.4 Conclusion Validity

The corpus is deterministic, not adversarial, so reported effectiveness characterises performance on a published taxonomy of known misconfiguration classes; no claim is made regarding novel or deliberately obfuscated insecure configurations, or an adversary who has read the policy set. On statistical conclusion validity, the $d_{\min}$ column of Table 6 bounds what the paired tests could establish, and three of the ten comparisons could not have reached significance under any outcome. We therefore draw no conclusion of equivalence anywhere in this paper, and the effectiveness comparison among scanners should be read as unresolved rather than as settled in either direction.

8 Replication Package

The replication package contains the benchmark corpus with ground-truth annotations and the generator specifications from which those annotations derive; the canonical control taxonomy as auditable data; the per-case output of the blind relabelling pass of Section 4.3, with its reasons and its stated limitations; the execution harness, which records raw tool output, resolved versions, and repeated latency samples; the normalization engine; the statistical implementation, with a test suite pinning each method to independently derived values; and the analysis

stage that generates every table in this paper. It also retains the measured results from before the plan-level policy correction disclosed in Section 7.2, so that the disclosure can be checked rather than taken on trust.

Three properties are worth stating explicitly. The harness declines to emit results when no case is admissible, instead of producing an empty but plausible table. The statistical implementation refuses to report a proportion whose denominator is zero, returning a not-estimable marker instead of a value a reader could mistake for a measurement. And both figures in this paper are generated from the recorded results rather than drawn, with a continuous-integration check that fails if a re-measurement moves a corpus size or tool version that a figure asserts.

Fig. 3 gives the package layout. Ground-truth labels, the scenario corpus, and the policy sources are separate top-level artefacts, so a reader can substitute an independently authored label set without modifying the harness.

9 Conclusion

This paper contributes an evaluation methodology for IaC security validation tools: a finding-normalization scheme that makes heterogeneous scanner output commensurable, an explicit treatment of matching strictness, a mechanically enforced corpus admissibility procedure, and a statistical protocol suited to paired small-sample comparison. The accompanying framework composes repository-edge validation, native module testing, and plan-level policy evaluation, and is released with a harness in which every reported figure is a generated artefact traceable to recorded tool output.

The methodological findings are the transferable result. On this corpus the matching criterion moves reported recall by as much as the entire spread between the tools compared, and enough to reorder them, which means comparative claims in this area should report their criterion as part of their method. Taxonomy coverage exerts a bias in the same direction as its author. And corpus size is a weaker indicator of evidential strength than admissibility, interval width, and the smallest difference the chosen test could have resolved [Böhme et al., 2022].

One outcome bears restating. Applied to the pipeline this paper contributes, the procedure reports that a third-party scanner attains the highest point-estimate control-level recall on the corpus and that the pipeline's scored layers do not lead the comparison. Reported

```
benchmark/
  benchmark.json          case catalogue + labels
  generate_corpus.py      case generator
  internal/               authored cases
  external/               CIS-derived cases
  labelling/              blind second pass
evaluation/
  corpus.py               admissibility gate
  normalize.py            finding normalization
  control_map.json        canonical taxonomy
  stats.py                exact CI + McNemar
  tables.py               LaTeX table emitters
  analyze.py              analysis entry point
security_framework/
  policies/*.rego         plan-level policy set
  engine/                 layer 1 scanning
infrastructure/
  tests/*.tftest.hcl      native module suite
results/
  raw/                    per-case tool output
  run_manifest.json       resolved versions
  tables/                 generated tables
  pre_opa_polarity_fix/   pre-correction run
experiments/
  run_baselines.sh        full re-measurement
  generate_figures.py     figure sources
```

Figure 3: Replication package layout. Ground-truth labels, the case corpus, and the policy sources are separate top-level artefacts, so an independently authored label set can be substituted without modifying the harness. The `pre_opa_polarity_fix` directory retains the measurements taken before the policy correction disclosed in Section 7.2.

without adjustment, that is one indication the procedure is not self-flattering; it is not proof of the procedure's neutrality, since Section 7.2 discloses a tuning asymmetry running the other way.

Three lines of future work follow directly. A mapping between the security-smell taxonomy that existing manually annotated IaC oracles are labelled against and the CIS control identifiers this evaluation uses would allow effectiveness to be measured on real-world configurations the investigator did not author, which is the mitigation Section 7.1 identifies as outstanding and the single change that would most strengthen the evidence. Enlarging the compliant baseline would narrow the specificity intervals to a width at which false-positive behaviour can be compared rather than merely reported. And extending the taxonomy beyond Terraform would establish whether the criterion sensitivity measured here is a property of IaC scanning or of this

technology.

Declarations

Funding This research received no external funding.

Competing interests The author declares no competing interests. The author is not affiliated with, and receives no support from, the vendor or maintainer of any tool evaluated in this study.

Ethics approval Not applicable. This study analyses software artefacts and involves no human participants or personal data.

Consent to participate Not applicable.

Consent for publication Not applicable.

Data and code availability The corpus, ground-truth annotations, canonical control taxonomy, normalization engine, execution harness, statistical implementation, analysis code, and the raw per-case tool output from which every table in this paper is generated are openly available:

`https://github.com/mchittineni/IaCSecBench`
`https://doi.org/10.5281/zenodo.21645016`

Author contributions The sole author designed the study, implemented the harness and the normalization engine, conducted the measurements, performed the analysis, and wrote the manuscript.

Use of AI-assisted technologies A large language model was used as the second rater in the blind relabelling pass of Section 4.3. That use is methodological and is described there, including its limitations and its effect on the reported agreement; the resulting per-case labels and reasons are in the replication package. A language model was additionally used for copy-editing of the manuscript text, which improves readability without generating content or making editorial decisions. All technical content, measurements, analysis and conclusions are the author's, who accepts full responsibility for the manuscript.

Table 9: STRIDE categories, mitigations, and the layer that enforces them.

Category	**Mitigation**	**Layer**
Spoofing	Authenticated API authorisers; transport encryption enforcement; prohibition of embedded credentials	1, 3
Tampering	Schema validation; pre-commit enforcement; evaluation of compiled plan documents	1, 2
Repudiation	Structured pipeline telemetry; recorded test execution; compliance tagging	2, 3
Information disclosure	Pattern-based personal-data detection; mandatory encryption at rest; storage public-access blocking	1, 3
Denial of service	Validation of web application firewall association, header handling, and request throttling	2, 3
Elevation of privilege	Policy denial of wildcard identity actions and unrestricted ingress ranges	3

A Scope of the Validated Threats

The controls the pipeline enforces were selected against a STRIDE decomposition [Shostack, 2014] of a continuous-integration deployment path, summarised in Table 9. Protected assets are infrastructure configurations and compiled plan states, ingested datasets, policy and test definitions, and pipeline credentials; the trust boundaries separate the developer workspace from version control, the evaluation runtime from the target repository, and the compiled plan from the policy engine.

We record the decomposition to delimit which misconfiguration classes the corpus exercises, and claim nothing further from it. The pipeline was not subjected to adversarial testing and no adversary model was validated, so the table states intended coverage rather than a result. Readers seeking a threat model of IaC pipelines as such should look to dedicated work [Shamim et al., 2020].

Four domains are out of scope: post-deployment runtime compromise, including container and hypervisor vulnerabilities; supply-chain compromise of provider binaries or registry mirrors; IaC technologies other than Terraform, since the plan-level layer presupposes Terraform plan JSON; and physical and social-engineering attacks.

References

Alan Agresti and Brent A. Coull. Approximate is better than "exact" for interval estimation of binomial proportions. *The American Statistician*, 52(2):119–126, 1998. doi: 10.1080/00031305.1998.10480550.

Andrea Arcuri and Lionel Briand. A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering. *Software Testing, Verification and Reliability*, 24(3):219–250, 2014. doi: 10.1002/stvr.1486.

Michael Armbrust, Armando Fox, Rean Griffith, Anthony D. Joseph, Randy Katz, Andy Konwinski, Gunho Lee, David Patterson, Ariel Rabkin, Ion Stoica, and Matei Zaharia. A view of cloud computing. *Communications of the ACM*, 53(4):50–58, 2010. doi: 10.1145/1721654.1721672.

Len Bass, Ingo Weber, and Liming Zhu. *DevOps: A Software Architect's Perspective*. Addison-Wesley, Boston, MA, USA, 2015.

Marcel Böhme, László Szekeres, and Jonathan Metzman. On the reliability of coverage-based fuzzer benchmarking. In *Proc. 44th Int. Conf. Software Engineering (ICSE)*, pages 1621–1633, 2022. doi: 10.1145/3510003.3510230.

Center for Internet Security. CIS Amazon Web Services foundations benchmark, v3.0.0. `https://www.cisecurity.org/benchmark/amazon_web_services`, 2024. accessed Aug. 2, 2026.

Michele Chiari, Michele De Pascalis, and Matteo Pradella. Static analysis of infrastructure as code: A survey. In *Proc. IEEE 19th Int. Conf. Software Architecture Companion (ICSA-C)*, pages 218–225, 2022. doi: 10.1109/ICSA-C54293.2022.00049.

C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4): 404–413, 1934. doi: 10.1093/biomet/26.4.404.

Stefano Dalla Palma, Dario Di Nucci, and Damian A. Tamburri. Toward a catalog of software quality metrics for infrastructure code. *Journal of Systems and Software*, 170:110726, 2020. doi: 10.1016/j.jss.2020.110726.

Stefano Dalla Palma, Dario Di Nucci, Fabio Palomba, and Damian A. Tamburri. Within-project defect prediction of infrastructure-as-code

using product and process metrics. *IEEE Trans. Software Engineering*, 48(6):2086–2104, 2022. doi: 10.1109/TSE.2021.3051492.

Thomas G. Dietterich. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923, 1998. doi: 10.1162/089976698300017197.

Matteo Esposito, Valentina Falaschi, and Davide Falessi. An extensive comparison of static application security testing tools. In *Proc. 28th Int. Conf. Evaluation and Assessment in Software Engineering (EASE)*, pages 69–78, 2024. doi: 10.1145/3661167.3661199.

Patrick Loic Foalem, Leuson Da Silva, Foutse Khomh, and Ettore Merlo. An empirical study of policy-as-code adoption in open-source software projects. *Journal of Systems and Software*, 242:113028, 2026. doi: 10.1016/j.jss.2026.113028.

Xiuting Ge, Chunrong Fang, Xuanye Li, Weisong Sun, Daoyuan Wu, Juan Zhai, Shang-Wei Lin, Zhihong Zhao, Yang Liu, and Zhenyu Chen. Machine learning for actionable warning identification: A comprehensive survey. arXiv preprint arXiv:2312.00324, 2023.

Anshuman B. Gokhale, Vaishnavi Jayaprakash, and Nagasundari Singaravelu. Comparative analysis of infrastructure-as-code misconfiguration detection through policy driven evaluation and taint-aware static reasoning. In *Proc. Int. Conf. Computing, Communication and Networking Technologies (Lecture Notes in Networks and Systems)*, pages 400–413, 2026. doi: 10.1007/978-3-032-27160-0_37.

Michele Guerriero, Martin Garriga, Damian A. Tamburri, and Fabio Palomba. Adoption, support, and challenges of infrastructure-as-code: Insights from industry. In *Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME)*, pages 580–589, 2019. doi: 10.1109/ICSME.2019.00092.

Andrew Habib and Michael Pradel. How many of all bugs do we find? a study of static bug detectors. In *Proc. 33rd IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, pages 317–328, 2018. doi: 10.1145/3238147.3238213.

HashiCorp. Terraform tests. `https://developer.hashicorp.com/terraform/language/tests`, 2024. accessed Aug. 2, 2026.

Wilhelm Hasselbring. Benchmarking as empirical standard in software engineering research. In *Proc. 25th Int. Conf. Evaluation and Assessment in Software Engineering (EASE)*, pages 365–372, 2021. doi: 10.1145/3463274.3463361.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. `https://www.jstor.org/stable/4615733`.

Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, Boston, MA, USA, 2010.

Ioanna Angeliki Kapetanidou, Alexandros Nizamis, and Konstantinos Votis. An evaluation of commonly used Kubernetes security scanning tools. In *Proc. 2nd Int. Workshop on MetaOS for the Cloud-Edge-IoT Continuum (MECC), companion of EuroSys*, pages 20–25, 2025. doi: 10.1145/3721889.3721924.

Indika Kumara, Martín Garriga, Angel Urbano Romeu, Dario Di Nucci, Fabio Palomba, Damian Andrew Tamburri, and Willem-Jan van den Heuvel. The do's and don'ts of infrastructure code: A systematic gray literature review. *Information and Software Technology*, 137:106593, 2021. doi: 10.1016/j.infsof.2021.106593.

Kaixuan Li, Sen Chen, Lingling Fan, Ruitao Feng, Han Liu, Chengwei Liu, Yang Liu, and Yixiang Chen. Comparison and evaluation on static application security testing (sast) tools for Java. In *Proc. 31st ACM Joint European Software Engineering Conf. and Symp. Foundations of Software Engineering (ESEC/FSE)*, pages 921–933, 2023. doi: 10.1145/3611643.3616262.

B. W. Matthews. Comparison of the predicted and observed secondary structure of T4 phage lysozyme. *Biochimica et Biophysica Acta (BBA) – Protein Structure*, 405(2):442–451, 1975. doi: 10.1016/0005-2795(75)90109-9.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.

Francesco Minna, Agathe Blaise, Katja Tuma, and Fabio Massacci. Automated analysis of security policy violations in Helm charts. *IEEE Trans. Dependable and Secure Computing*, 23(2):2519–2533, 2026. doi: 10.1109/TDSC.2025.3628213.

Kief Morris. *Infrastructure as Code: Dynamic Systems for the Cloud Age*. O'Reilly Media, Sebastopol, CA, USA, 2nd edition, 2020.

National Institute of Standards and Technology. Security and privacy controls for information systems and organizations. Technical Report Special Publication 800-53, Rev. 5, NIST, 2020.

Ruben Opdebeeck, Ahmed Zerouali, and Coen De Roover. Smelly variables in Ansible infrastructure code: Detection, prevalence, and lifetime. In *Proc. 19th Int. Conf. Mining Software Repositories (MSR)*, pages 61–72, 2022. doi: 10.1145/3524842.3527964.

Ruben Opdebeeck, Ahmed Zerouali, and Coen De Roover. Control and data flow in security smell detection for infrastructure as code: Is it worth the effort? In *Proc. 20th IEEE/ACM Int. Conf. Mining Software Repositories (MSR)*, pages 534–545, 2023. doi: 10.1109/MSR59073.2023.00079.

Open Policy Agent. Open Policy Agent documentation. `https://www.openpolicyagent.org/docs`, 2024. accessed Aug. 2, 2026.

OWASP Foundation. OWASP benchmark project. `https://owasp.org/www-project-benchmark/`, 2023. accessed Aug. 2, 2026.

Yiming Qiu, Patrick Tser Jern Kon, Ryan Beckett, and Ang Chen. Unearthing semantic checks for cloud infrastructure-as-code programs. In *Proc. 30th ACM Symp. Operating Systems Principles (SOSP)*, pages 574–589, 2024. doi: 10.1145/3694715.3695974.

Akond Rahman, Rezvan Mahdavi-Hezaveh, and Laurie Williams. A systematic mapping study of infrastructure as code research. *Information and Software Technology*, 108:65–77, 2019a. doi: 10.1016/j.infsof.2018.12.004.

Akond Rahman, Chris Parnin, and Laurie Williams. The seven sins: Security smells in infrastructure as code scripts. In *Proc. 41st Int. Conf. Software Engineering (ICSE)*, pages 164–175, Montreal, QC, Canada, 2019b. doi: 10.1109/ICSE.2019.00033.

Akond Rahman, Effat Farhana, Chris Parnin, and Laurie Williams. Gang of eight: A defect taxonomy for infrastructure as code scripts. In *Proc. 42nd Int. Conf. Software Engineering (ICSE)*, pages 752–764, 2020. doi: 10.1145/3377811.3380409.

Akond Rahman, Md Rayhanur Rahman, Chris Parnin, and Laurie Williams. Security smells in Ansible and Chef scripts: A replication study. *ACM Trans. Software Engineering and Methodology*, 30(1):1–31, 2021. doi: 10.1145/3408897.

Akond Rahman, Shazibul Islam Shamim, Dibyendu Brinto Bose, and Rahul Pandita. Security misconfigurations in open source Kubernetes manifests: An empirical study. *ACM Trans. Software Engineering and Methodology*, 32(4):1–36, 2023. doi: 10.1145/3579639.

Paul Ralph. ACM SIGSOFT empirical standards released. *ACM SIGSOFT Software Engineering Notes*, 46(1):19, 2021. doi: 10.1145/3437479.3437483.

Harry Roe, Mandar Gogate, and Kia Dashtipour. Multicloud security assessment: A benchmark study of infrastructure as code scanners. *ICCK Trans. Information Security and Cryptography*, 2(2):109–118, 2026. doi: 10.62762/tisc.2026.777114.

Nuno Saavedra and João F. Ferreira. GLITCH: Automated polyglot security smell detection in infrastructure as code. In *Proc. 37th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, pages 1–12, 2022. doi: 10.1145/3551349.3556945.

Nuno Saavedra, João Gonçalves, Miguel Henriques, João F. Ferreira, and Alexandra Mendes. Polyglot code smell detection for infrastructure as code with GLITCH. In *Proc. 38th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, pages 2042–2045, 2023. doi: 10.1109/ASE56229.2023.00162.

Md Shazibul Islam Shamim, Farzana Ahamed Bhuiyan, and Akond Rahman. XI commandments of Kubernetes security: A systematization of knowledge related to Kubernetes security practices. In *Proc. IEEE Secure Development Conf. (SecDev)*, pages 58–64, 2020. doi: 10.1109/SecDev45635.2020.00025.

Tushar Sharma, Marios Fragkoulis, and Diomidis Spinellis. Does your configuration code smell? In *Proc. 13th Int. Conf. Mining Software Repositories (MSR)*, pages 189–200, 2016. doi: 10.1145/2901739.2901761.

Adam Shostack. *Threat Modeling: Designing for Security*. John Wiley & Sons, Indianapolis, IN, USA, 2014.

Mohammad Tahaei, Kami Vaniea, Konstantin (Kosta) Beznosov, and Maria K Wolters. Security notifications in static analysis tools: Developers' attitudes, comprehension, and ability to act on them. In *Proc. ACM CHI Conf. Human Factors in Computing Systems*, pages 1–17, 2021. doi: 10.1145/3411764.3445616.

Carmine Vassallo, Sebastiano Panichella, Fabio Palomba, Sebastian Proksch, Harald C. Gall, and Andy Zaidman. How developers engage with static analysis tools in different contexts. *Empirical Software Engineering*, 25(2):1419–1457, 2019. doi: 10.1007/s10664-019-09750-5.

Alexandre Verdet, Mohammad Hamdaqa, Leuson Da Silva, and Foutse Khomh. Exploring security practices in infrastructure as code: An empirical study. arXiv preprint arXiv:2308.03952, 2023.

Alexandre Verdet, Mohammad Hamdaqa, Leuson Da Silva, and Foutse Khomh. Assessing the adoption of security policies by developers in Terraform across different cloud providers. *Empirical Software Engineering*, 30(2), 2025. doi: 10.1007/s10664-024-10610-0.

Aicha War, Serge Lionel Nikiema, Jordan Samhi, Jacques Klein, and Tegawendé F. Bissyandé. Security smells in infrastructure as code: A taxonomy update beyond the seven sins. arXiv preprint arXiv:2509.18761, 2025.

Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, Berlin, Germany, 2012. doi: 10.1007/978-3-642-29044-2.